



APB-FIFO Verification Bug Report

UVM Testbench Implementation

Project Information

Design: APB3-Accessible Synchronous FIFO
Width: 8 bits
Depth: 16 entries
Protocol: AMBA APB3
Verification: UVM 1.2

Student Information

Name: Osaïd Nur
Student ID: 1210733
Department: Electrical and Computer Engineering
Course: Hardware Design Lab ENCS5131
Section: 1

January 14, 2026

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Design Specifications	3
1.3	Register Map	3
2	UVM Verification Environment	3
2.1	Testbench Architecture	3
2.1.1	UVM Component Hierarchy	4
2.1.2	Component Descriptions	4
3	APB Protocol Implementation	5
3.1	APB3 Protocol Overview	5
3.2	APB Driver Implementation	5
4	Test Suite	6
4.1	Reset Test	6
4.2	Enable Test	6
4.3	Clear Test	6
4.4	Basic Operation Test	6
4.5	Overflow Test	7
4.6	Underflow Test	7
4.7	Threshold Test	7
4.8	Random Test	8
4.9	Stress Test	8
5	Bug Reports	8
5.1	Bug Summary	8
5.2	BUG-001: Reset Logic Inversion	8
5.2.1	Description	8
5.2.2	Test Scenario	8
5.2.3	Actual Behavior from sim.log	9
5.2.4	Analysis	9
5.3	BUG-002: Overflow Flag Not Sticky	9
5.3.1	Description	9
5.3.2	Test Scenario	10
5.3.3	Actual Behavior from sim.log	10
5.3.4	Analysis	10
5.4	BUG-003: DROP_ON_FULL Feature Not Functional	11
5.4.1	Description	11
5.4.2	Test Scenario	11
5.4.3	Actual Behavior from sim.log	12
5.4.4	Analysis	12
5.5	BUG-004: Underflow Flag Not Sticky	12
5.5.1	Description	12
5.5.2	Test Scenario	13
5.5.3	Actual Behavior from sim.log	13
5.5.4	Analysis	13

5.6	BUG-005: Full Flag Threshold Error	14
5.6.1	Description	14
5.6.2	Test Scenario	14
5.6.3	Actual Behavior from sim.log	15
5.6.4	Analysis	15

1 Introduction

1.1 Project Overview

This document presents a comprehensive bug report for an APB3-accessible synchronous FIFO design. The FIFO was verified using a UVM (Universal Verification Methodology) testbench, which successfully identified multiple critical bugs in the RTL implementation.

1.2 Design Specifications

The APB FIFO design under test has the following specifications:

- **Data Width:** 8 bits
- **FIFO Depth:** 16 entries
- **Interface Protocol:** AMBA APB3 (Advanced Peripheral Bus)
- **Reset Type:** Active-low asynchronous reset
- **Features:**
 - Programmable almost-full and almost-empty thresholds
 - Overflow and underflow detection (sticky flags)
 - Drop-on-full mode (configurable)
 - Synchronous clear functionality

1.3 Register Map

The FIFO is accessible through four APB-mapped registers:

Offset	Name	Access	Description
0x00	CTRL	RW	Control register (Enable, Clear, Drop-on-full)
0x04	THRESH	RW	Threshold configuration (almost-full, almost-empty)
0x08	STATUS	RO	Status flags and FIFO count
0x0C	DATA	RW	FIFO data (push on write, pop on read)

Table 1: APB FIFO Register Map

2 UVM Verification Environment

2.1 Testbench Architecture

The verification environment follows the standard UVM architecture with the following components:

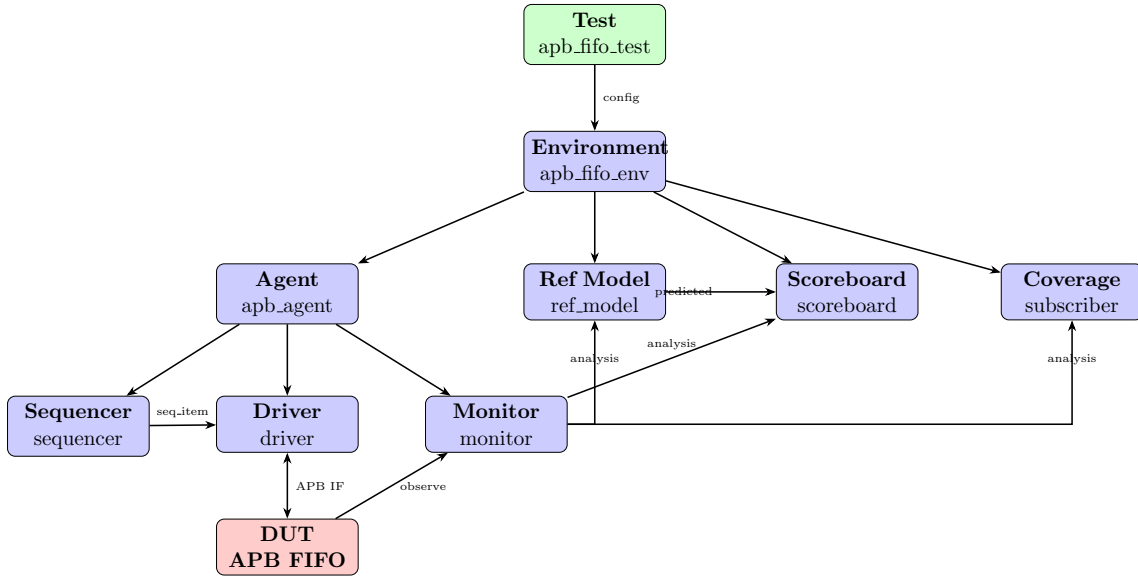


Figure 1: UVM Testbench Architecture

2.1.1 UVM Component Hierarchy

2.1.2 Component Descriptions

- **Sequence Item (apb_sequence_item):** Represents a single APB transaction with randomization constraints for valid register addresses and data fields.
- **Sequencer (apb_sequencer):** Standard UVM sequencer for managing sequence items.
- **Driver (apb_driver):** Implements the APB3 protocol:
 - SETUP phase: Assert PSEL, set address and control signals
 - ACCESS phase: Assert PENABLE, wait for PREADY
 - Handles zero-wait-state and multi-cycle transfers
- **Monitor (apb_monitor):** Observes APB transactions and broadcasts them via analysis port
- **Agent (apb_agent):** Contains sequencer, driver, and monitor. Configured as active agent.
- **Reference Model (apb_fifo_ref_model):** Golden reference model that mimics expected FIFO behavior:
 - Implements push/pop operations
 - Tracks FIFO count and status flags
 - Handles threshold comparisons
 - Manages overflow/underflow conditions
- **Scoreboard (apb_fifo_scoreboard):** Compares DUT responses with reference model predictions
- **Coverage Subscriber:** Collects functional coverage data

3 APB Protocol Implementation

3.1 APB3 Protocol Overview

The Advanced Peripheral Bus (APB3) is a simple, low-power bus protocol designed for peripheral access. It operates with a two-cycle transfer mechanism:

1. **SETUP Phase:** The master asserts PSEL and sets up address (PADDR), write/read control (PWRITE), and write data (PWDATA if write). PENABLE remains low.
2. **ACCESS Phase:** PENABLE is asserted. The slave responds with PREADY when ready, and provides read data (PRDATA if read) or error status (PSLVERR).
3. **IDLE Phase:** After PREADY is received, PSEL and PENABLE return to low, completing the transfer.

3.2 APB Driver Implementation

The driver implements the standard APB3 protocol as shown below:

```

1 task drive(apb_sequence_item tr);
2   // Wait until reset is deasserted
3   while (vif.PRESETn !== 1'b1) begin
4       drive_idle();
5       @(vif.drv_cb);
6   end
7
8   // SETUP cycle
9   @(vif.drv_cb);
10  vif.drv_cb.PSEL      <= 1'b1;
11  vif.drv_cb.PENABLE  <= 1'b0;
12  vif.drv_cb.PWRITE   <= tr.pwrite;
13  vif.drv_cb.PADDR    <= tr.paddr;
14  vif.drv_cb.PWDATA   <= tr.pwdata;
15
16  // ACCESS cycle
17  @(vif.drv_cb);
18  vif.drv_cb.PENABLE  <= 1'b1;
19
20  // Wait for PREADY
21  while (vif.drv_cb.PREADY !== 1'b1) begin
22      @(vif.drv_cb);
23  end
24
25  // Capture response
26  tr.pslverr = vif.drv_cb.PSLVERR;
27  if (!tr.pwrite) begin
28      tr.prdata = vif.drv_cb.PRDATA;
29  end
30
31  // Return to IDLE
32  @(vif.drv_cb);
33  drive_idle();
34 endtask

```

Listing 1: APB Driver Transaction Method

4 Test Suite

The verification suite includes 12 directed and random tests designed to exercise all FIFO functionality and corner cases.

4.1 Reset Test

Objective: Verify reset behavior

Sequence: reset_sequence

Coverage:

- Enable FIFO
- Push data into FIFO
- Read status flags(empty,count)

4.2 Enable Test

Objective: Verify FIFO enable functionality

Sequence: fifo_enable_sequence

Coverage:

- Enable and Disable FIFO
- Attempt push/pop operations
- Read Status flags

4.3 Clear Test

Objective: Verify FIFO clear functionality

Sequence: fifo_clear_sequence

Coverage:

- Push data
- Clear FIFO
- Verify empty and full state (Corner Cases)

4.4 Basic Operation Test

Objective: Verify basic push and pop operations

Sequence: basic_push_pop_sequence

Coverage:

- Enable FIFO
- Push random data values
- Verify FIFO count increases

- Pop random data values
- Verify data integrity (FIFO ordering)
- Verify FIFO returns to empty state
- Fill FIFO element by element to capacity (16 entries)
- Monitor full flag transition at maximum capacity
- Verify FIFO can be cleared and reused

4.5 Overflow Test

Objective: Verify overflow detection

Sequence: overflow_sequence

Coverage:

- Fill FIFO to capacity (16 entries)
- Attempt additional push
- Verify overflow flag assertion
- Test drop-on-full mode

4.6 Underflow Test

Objective: Verify underflow detection

Sequence: underflow_sequence

Coverage:

- Start with empty FIFO
- Attempt pop operation
- Verify underflow flag assertion

4.7 Threshold Test

Objective: Verify programmable threshold functionality

Sequence: threshold_sequence

Coverage:

- Configure almost-full threshold (e.g., 10)
- Configure almost-empty threshold (e.g., 3)
- Push data and monitor almost-empty flag transition
- Continue pushing and monitor almost-full flag transition

4.8 Random Test

Objective: Random stress testing

Sequence: random_sequence

Coverage: Random mix of all operations

4.9 Stress Test

Objective: Back-to-back execution of all sequences

Test: stress_test

Coverage:

- 5 iterations of 200 random transactions (1000 total operations)
- Run all sequences consecutively in a single test
- Sustained operation stability verification

5 Bug Reports

5.1 Bug Summary

During verification, five critical bugs were identified in the RTL design. All bugs were discovered through UVM testbench execution and confirmed through detailed analysis.

Bug ID	Description	Severity
BUG-001	Reset Logic Inversion	Critical
BUG-002	Overflow Flag Not Sticky	Critical
BUG-003	DROP_ON_FULL Feature Not Functional	Critical
BUG-004	Underflow Flag Not Sticky	Critical
BUG-005	Full Flag Threshold Error	Critical

Table 2: Bug Summary Table

5.2 BUG-001: Reset Logic Inversion

Severity: CRITICAL

Test: reset_test

5.2.1 Description

The FIFO does not clear properly when hardware reset (PRESETn) is asserted and deasserted. The FIFO appears to reset on the wrong edge of PRESETn signal.

5.2.2 Test Scenario

The `reset_test` sequence performs the following steps:

1. Assert reset ($\text{PRESETn} = 0$) → read status

2. Enable FIFO
3. Push 3 data items into fifo → **read status**
4. Assert reset again (PRESETn = 0) → **read status**
5. Suppose the opposite polarity is used, repeat the previous steps with PRESETn = 1 when reset → **read status**

5.2.3 Actual Behavior from sim.log

```

1 UVM_INFO [reset_sequence] Pushed 3 data items to FIFO
2 UVM_INFO [reset_sequence] Asserting hardware reset again (PRESETn=0)
3 UVM_ERROR [Scoreboard]
=====
4 UVM_ERROR [Scoreboard] STATUS Mismatch: Expected=0x000000c0, Actual=0
  x00000009
5 UVM_ERROR [Scoreboard] STATUS Details:
6 UVM_ERROR [Scoreboard]   Empty:      Exp=0 Act=1 MISMATCH
7 UVM_ERROR [Scoreboard]   Full:       Exp=0 Act=0
8 UVM_ERROR [Scoreboard]   AlmostFull: Exp=0 Act=0
9 UVM_ERROR [Scoreboard]   AlmostEmpty: Exp=0 Act=1 MISMATCH
10 UVM_ERROR [Scoreboard]   Overflow:   Exp=0 Act=0
11 UVM_ERROR [Scoreboard]   Underflow:  Exp=0 Act=0
12 UVM_ERROR [Scoreboard]   Count:     Exp=3 Act=0 MISMATCH
13 UVM_ERROR [Scoreboard]
=====

```

5.2.4 Analysis

Expected Behavior:

- after pushing data: count should be 3, empty flag should clear
- After reset assertion (PRESETn=0): FIFO should be cleared and ready for normal operation

Actual Behavior:

- Push operations have no effect (count stays 0, empty stays 1)
- FIFO appears to clear when PRESETn is deasserted (goes high)
- After deassertion, FIFO remains stuck in reset state

5.3 BUG-002: Overflow Flag Not Sticky

Severity: **CRITICAL**

Test: overflow_test

5.3.1 Description

The overflow flag does not behave as a sticky flag. According to the design specification, the overflow flag should remain high once set until explicitly cleared by writing to the CTRL register. However, testing reveals that the overflow flag gets cleared unexpectedly.

5.3.2 Test Scenario

The `overflow_test` runs the `overflow_sequence`, which tests the sticky flag property through the following steps:

1. Clear and enable the FIFO with `DROP_ON_FULL` disabled
2. Fill the FIFO completely to capacity (16 elements)
3. Push one additional element to trigger overflow condition
4. Verify overflow flag is set
5. Pop 3 elements from the FIFO
6. Read status register - overflow flag should still be high (sticky behavior)
7. Continue with additional push operations
8. Verify overflow flag remains set throughout normal operations

The test specifically validates that overflow flag persists across multiple operations until software explicitly clears it.

5.3.3 Actual Behavior from `sim.log`

```

1 UVM_INFO [overflow_sequence] Popped 3 elements - overflow flag should
  remain set
2 UVM_INFO [Scoreboard] DATA Match: 0x00000002
3 UVM_INFO [Scoreboard] RefModel State: EN=1, Count=13/16, Empty=0, Full
  =0, AF=0, AE=0, OVF=1, UNF=0
4 UVM_ERROR [Scoreboard]
  =====
5 UVM_ERROR [Scoreboard] STATUS Mismatch: Expected=0x00000350, Actual=0
  x00000340
6 UVM_ERROR [Scoreboard] STATUS Details:
7 UVM_ERROR [Scoreboard]   Empty:      Exp=0 Act=0
8 UVM_ERROR [Scoreboard]   Full:       Exp=0 Act=0
9 UVM_ERROR [Scoreboard]   AlmostFull: Exp=0 Act=0
10 UVM_ERROR [Scoreboard]  AlmostEmpty: Exp=0 Act=0
11 UVM_ERROR [Scoreboard]   Overflow:   Exp=1 Act=0 MISMATCH
12 UVM_ERROR [Scoreboard]   Underflow:  Exp=0 Act=0
13 UVM_ERROR [Scoreboard]   Count:      Exp=13 Act=13
14 UVM_ERROR [Scoreboard]
  =====

```

5.3.4 Analysis

Expected Behavior:

- After overflow is triggered, the flag should remain set (`OVF=1`)
- The reference model shows the overflow flag should stay high (`OVF=1` in RefModel State)

- The flag should only clear when software explicitly writes to the CTRL register's clear bit

Actual Behavior:

- After some operations (pushing, popping), the test reads the status register
- The reference model correctly expects OVF=1 (overflow sticky bit should still be set)
- However, when reading the status register, the DUT returns OVF=0 (overflow flag is cleared)
- The overflow flag is not maintaining its sticky behavior as specified
- This creates a mismatch: Expected OVF=1, Actual OVF=0

5.4 BUG-003: DROP_ON_FULL Feature Not Functional

Severity: CRITICAL**Test:** overflow_test

5.4.1 Description

The DROP_ON_FULL control bit does not function as specified. When enabled, this bit should cause the FIFO to return an APB error response (PSLVERR=1) when attempting to write to a full FIFO. However, the DUT never asserts PSLVERR regardless of the DROP_ON_FULL setting.

5.4.2 Test Scenario

The `overflow_test` runs the `overflow_sequence`, which includes Test 2 to verify DROP_ON_FULL functionality:

1. Clear the FIFO
2. Enable the FIFO with DROP_ON_FULL=1 (write CTRL register = 0x5)
3. Fill the FIFO to full capacity (16 elements)
4. Verify FIFO is full
5. Attempt to push additional element to full FIFO
6. Expected: PSLVERR should assert (error response on APB bus)
7. Verify overflow flag is set
8. Attempt another push to full FIFO
9. Expected: PSLVERR should assert again

The test specifically validates that when DROP_ON_FULL is enabled, writes to a full FIFO generate APB error responses.

5.4.3 Actual Behavior from sim.log

```

1 UVM_INFO [overflow_sequence] --- Test 2: Overflow with DROP_ON_FULL
   Enabled ---
2 UVM_INFO [overflow_sequence] FIFO cleared
3 UVM_INFO [overflow_sequence] FIFO enabled with DROP_ON_FULL=1
4 UVM_INFO [overflow_sequence] Filling FIFO to capacity (16 elements)
5 UVM_INFO [overflow_sequence] FIFO filled to capacity
6 UVM_INFO [Scoreboard] STATUS Match: 0x00000406
7 UVM_INFO [Scoreboard] RefModel State: EN=1, Count=16/16, Empty=0, Full
   =1, AF=1, AE=0, OVF=0, UNF=0
8 UVM_INFO [overflow_sequence] Attempted to push extra element to full
   FIFO (should drop error)
9 UVM_ERROR [Scoreboard]
   =====
10 UVM_ERROR [Scoreboard] PSLVERR Mismatch: Expected=1, Actual=0
11 UVM_ERROR [Scoreboard]
   =====

```

5.4.4 Analysis

Expected Behavior:

- When DROP_ON_FULL=1 and FIFO is full, write attempts should be rejected
- The APB slave should assert PSLVERR=1 to signal the error to the master
- The reference model correctly expects PSLVERR=1 when pushing to full FIFO
- This provides immediate feedback to software that the write was dropped

Actual Behavior:

- FIFO correctly reaches full state (Full=1, count=16)
- DROP_ON_FULL bit is enabled (CTRL=0x5)
- When attempting to write to full FIFO, the test reads the APB response
- The reference model expects PSLVERR=1 (error response)
- However, the DUT returns PSLVERR=0 (no error signaled)
- The DROP_ON_FULL feature appears to have no effect on APB error signaling

5.5 BUG-004: Underflow Flag Not Sticky

Severity: CRITICAL

Test: underflow_test

5.5.1 Description

The underflow flag does not behave as a sticky flag. According to the design specification, the underflow flag should remain high once set until explicitly cleared by writing to the CTRL register. However, testing reveals that the underflow flag gets cleared unexpectedly during normal FIFO operations.

5.5.2 Test Scenario

The `underflow_test` runs the `underflow_sequence`, which tests the sticky flag property through the following steps:

1. Clear and enable the FIFO
2. Verify FIFO is empty (count=0)
3. Attempt to pop from empty FIFO to trigger underflow condition
4. Verify underflow flag is set
5. Push 3 elements into the FIFO
6. Pop 2 elements from the FIFO
7. Read status register - underflow flag should still be high (sticky behavior)
8. Verify underflow flag remains set throughout normal operations

The test specifically validates that underflow flag persists across multiple push and pop operations until software explicitly clears it.

5.5.3 Actual Behavior from sim.log

```

1 UVM_INFO [underflow_sequence] --- Testing Sticky Flag Property (
   underflow remains set) ---
2 UVM_INFO [underflow_sequence] Pushing 3 elements - underflow flag
   should remain set
3 UVM_INFO [Scoreboard] STATUS Match: 0x00000029
4 UVM_INFO [Scoreboard] RefModel State: EN=1, Count=0/16, Empty=1, Full
   =0, AF=0, AE=1, OVF=0, UNF=1
5 UVM_INFO [underflow_sequence] Popping 2 elements - underflow flag
   should remain set
6 UVM_ERROR [Scoreboard]
   =====
7 UVM_ERROR [Scoreboard] STATUS Mismatch: Expected=0x000000e0, Actual=0
   x000000c0
8 UVM_ERROR [Scoreboard] STATUS Details:
9 UVM_ERROR [Scoreboard] Empty: Exp=0 Act=0
10 UVM_ERROR [Scoreboard] Full: Exp=0 Act=0
11 UVM_ERROR [Scoreboard] AlmostFull: Exp=0 Act=0
12 UVM_ERROR [Scoreboard] AlmostEmpty: Exp=0 Act=0
13 UVM_ERROR [Scoreboard] Overflow: Exp=0 Act=0
14 UVM_ERROR [Scoreboard] Underflow: Exp=1 Act=0 MISMATCH
15 UVM_ERROR [Scoreboard] Count: Exp=3 Act=3
16 UVM_ERROR [Scoreboard]
   =====

```

5.5.4 Analysis

Expected Behavior:

- After underflow is triggered by popping from empty FIFO, the flag should remain set (UNF=1)

- The reference model shows the underflow flag should stay high (UNF=1 in RefModel State)
- The flag should only clear when software explicitly writes to the CTRL register's clear bit

Actual Behavior:

- After the underflow condition is triggered, some operations are performed (pushing and popping)
- The test then reads the status register
- The reference model correctly expects UNF=1 (underflow sticky bit should still be set)
- However, when reading the status register, the DUT returns UNF=0 (underflow flag is cleared)
- The underflow flag is not maintaining its sticky behavior as specified
- This creates a mismatch: Expected UNF=1, Actual UNF=0

5.6 BUG-005: Full Flag Threshold Error

Severity: CRITICAL**Test:** basic_operation_test

5.6.1 Description

The FIFO full flag asserts prematurely at count=15 (DEPTH-1) instead of count=16 (DEPTH). When the FIFO is treated as full at 15 entries, attempting to push the 16th element incorrectly triggers the overflow flag, even though the FIFO has not actually exceeded its capacity. As a consequence of this bug, the almost full flag will never raise because both the full flag and almost full flag are configured to assert at DEPTH-1 (count=15), creating a conflict where the full flag takes precedence and the almost full flag never gets the opportunity to assert.

5.6.2 Test Scenario

The `basic_operation_test` runs the `basic_push_pop_sequence`, which includes Test 3 that fills the FIFO element by element:

1. Clear and enable the FIFO
2. Fill FIFO element by element from 0 to FIFO_DEPTH (16 elements)
3. Read status register after each push operation
4. Monitor when empty, full, and overflow flags change state
5. Verify that full flag only asserts when all 16 entries are occupied
6. Verify that overflow only occurs when trying to push beyond capacity

The test validates that the FIFO correctly reports its full state and distinguishes between full and overflow conditions.

5.6.3 Actual Behavior from sim.log

```

1 UVM_INFO [Scoreboard] STATUS Match: 0x00000340
2 UVM_INFO [Scoreboard] RefModel State: EN=1, Count=13/16, Empty=0, Full
  =0, AF=0, AE=0, OVF=0, UNF=0
3 UVM_INFO [Scoreboard] STATUS Match: 0x00000380
4 UVM_INFO [Scoreboard] RefModel State: EN=1, Count=14/16, Empty=0, Full
  =0, AF=0, AE=0, OVF=0, UNF=0
5 UVM_ERROR [Scoreboard]
  =====
6 UVM_ERROR [Scoreboard] STATUS Mismatch: Expected=0x000003c0, Actual=0
  x000003c2
7 UVM_ERROR [Scoreboard] STATUS Details:
8 UVM_ERROR [Scoreboard] Empty: Exp=0 Act=0
9 UVM_ERROR [Scoreboard] Full: Exp=0 Act=1 MISMATCH
10 UVM_ERROR [Scoreboard] AlmostFull: Exp=0 Act=0
11 UVM_ERROR [Scoreboard] AlmostEmpty: Exp=0 Act=0
12 UVM_ERROR [Scoreboard] Overflow: Exp=0 Act=0
13 UVM_ERROR [Scoreboard] Underflow: Exp=0 Act=0
14 UVM_ERROR [Scoreboard] Count: Exp=15 Act=15
15 UVM_ERROR [Scoreboard]
  =====
16 UVM_INFO [basic_push_pop_sequence] FIFO filled to fifo depth: 16
17 UVM_ERROR [Scoreboard]
  =====
18 UVM_ERROR [Scoreboard] STATUS Mismatch: Expected=0x00000406, Actual=0
  x000003d2
19 UVM_ERROR [Scoreboard] STATUS Details:
20 UVM_ERROR [Scoreboard] Empty: Exp=0 Act=0
21 UVM_ERROR [Scoreboard] Full: Exp=1 Act=1
22 UVM_ERROR [Scoreboard] AlmostFull: Exp=1 Act=0 MISMATCH
23 UVM_ERROR [Scoreboard] Overflow: Exp=0 Act=1 MISMATCH
24 UVM_ERROR [Scoreboard] Underflow: Exp=0 Act=0
25 UVM_ERROR [Scoreboard] Count: Exp=16 Act=15 MISMATCH
26 UVM_ERROR [Scoreboard]
  =====

```

5.6.4 Analysis

Expected Behavior:

- At count=13 and count=14, the full flag should be low (Full=0)
- At count=15, the full flag should still be low (Full=0) - one more entry available
- At count=16, the full flag should assert (Full=1) - FIFO completely full
- Overflow should only occur when attempting to push beyond 16 entries

Actual Behavior:

- At count=13 and count=14, the full flag correctly stays low
- At count=15, the DUT incorrectly asserts the full flag (Full=1)
- The reference model expects Full=0 at count=15, but DUT shows Full=1

- When pushing the 16th element, the DUT sets overflow flag (OVF=1)
- The count shows 15 instead of 16, indicating the 16th push was rejected as overflow
- The FIFO is treating its capacity as 15 entries instead of 16